Summative Assessment Individual Presentation Transcript

Prepared for: Abdulrahman Husain Ahmed Alhashmi

Slide 1 — Title and framing

Good afternoon. I'm Abdulrahman Husain Ahmed Alhashmi My project focuses on object recognition using a compact convolutional neural network with residual blocks. The aim is simple: learn accurate class predictions from small, low-resolution colour images, while keeping the model efficient enough for constrained hardware. I will cover the dataset, data splits and validation, augmentation, the chosen architecture, hyperparameters, training controls, learning-curve interpretation, and final test results. I will also comment on why residual connections help in compact designs and where the trade-offs sit when balancing accuracy and speed.

Slide 2 — Problem statement and dataset overview

The task is multi-class image classification over ten everyday categories. We work with CIFAR-10: 60,000 RGB images at 32×32 pixels across ten balanced classes. Goals: High accuracy on unseen data; computational efficiency suitable for limited memory and real-time use. Why CIFAR-10: Balanced classes reduce bias in training signals; small image size speeds up iteration, letting us test ideas quickly. What success looks like: consistent validation gains, strong test performance without overfitting, and sensible inference time on standard GPUs and modest CPUs.

Slide 3 — Data splitting and validation setup

Split: 60k images into 45k training, 5k validation, 10k test. Rationale: The 5k validation set is reserved for hyperparameter tuning and early stopping; it guards against overfitting. Protocol: No augmentation or label leakage into validation/test; all tuning finalised before the single pass on the 10k test set. Benefits: Cleaner estimates of generalisation and fair comparison across settings; checkpoint selection by lowest validation loss.

Slide 4 — Pre-processing and augmentation

Normalisation: Per-channel mean/standard deviation computed on the training set to stabilise gradients and speed convergence. Augmentation on training only: random crop with padding, random horizontal flip, light colour jitter. Evaluation sets: validation and test receive only normalisation, not augmentation. Effect: Augmentations enlarge the effective dataset and reduce overfitting; clean evaluation preserves comparability across runs.

Slide 5 — Architecture choice: compact residual CNN

Design target: Preserve representational power with residual connections while keeping parameter count modest. Residual blocks: Identity shortcuts ease gradient flow and enable deeper stacks without vanishing gradients. Compactness tactics: fewer filters, careful down-sampling, and depthwise-separable convolutions where appropriate. Impact: Better convergence than a plain CNN of similar size; competitive accuracy per parameter and faster inference.

Slide 6 — Detailed components and flow

Stem: Initial Conv → BatchNorm → ReLU to extract low-level edges and colours. Body: Stacked residual blocks with Conv–BN–ReLU; down-sampling via strided convolutions at stage transitions to expand receptive field. Head: Global average pooling replaces large fully connected layers; a compact FC layer maps to ten logits; Softmax yields probabilities. Regularisation: Dropout around the classification head and L2 weight decay across the network. Why GAP: Fewer parameters, less overfitting, and better spatial invariance.

Slide 7 — Hyperparameters: choices and justification

Training length and batch size: ~100 epochs, batch size 128 for stable gradients and throughput. Optimiser: Adam for quick initial progress vs SGD with Nesterov momentum for stronger end-point generalisation; final runs use SGD-momentum guided by validation curves. Learning-rate schedule: Cosine decay to reduce step size smoothly. Regularisation: Weight decay 5×10^{-4}; Dropout ≈0.3 on the classifier. Principle: Start broad, track validation, then converge on the schedule with the smallest generalisation gap.

Slide 8 — Training controls and quality checks

Reproducibility: Fixed seeds for PyTorch, NumPy, and CUDA. Logging and checkpoints: Record loss/accuracy; save best model when validation loss reaches a new minimum. Early stopping: Patience of roughly 10–15 epochs to prevent over-training once validation loss stalls. Sanity checks: Augmentations apply only to training; first epochs should show falling training loss or we inspect LR, normalisation, or data pipeline. Outcome: A controlled environment that preserves the best-performing state.

Slide 9 — Reading the learning curves

Monitor: Training and validation accuracy rising together; losses declining without divergence. Interpreting gaps: Large train–val gap with low training loss suggests overfitting—strengthen regularisation or augmentation. Parallel but high losses indicate underfitting—consider more capacity or schedule adjustments. Convergence: With cosine decay, curves should flatten into a stable minimum rather than oscillate. If curves misbehave: Reduce LR or use warm-up; adjust weight decay or dropout; recalibrate augmentation intensity.

Slide 10 — Comparative Insights Across Approaches

Goal: address the 'multi-track' brief by placing the chosen CNN against Classical ML, Transfer Learning, and Self-Supervised/NAS.

- Classical ML (HOG/SIFT + SVM/KNN): quick to train, light on compute, but weaker on complex textures. Best when resources are tight or latency is critical on CPU.
- Compact CNN (this project): strong accuracy with modest parameters and fair latency. Good balance for CIFAR-10 scale and a clear training recipe.
- Transfer Learning (e.g., MobileNet/EfficientNet-B0): strong baseline in fewer epochs, yet needs careful resizing and fine-tuning to avoid overfitting to 32×32.
- Self-Supervised / NAS: promising when unlabelled data or search time is available, but adds setup complexity.

Take-away: for this assessment, the compact CNN offers the best trade-off between accuracy, training time, and reproducibility.

Slide 11 — Final test performance

On the 10k held-out test set, the compact residual CNN achieved:

- Accuracy: 92.5%
- Macro-Precision: 92.3%
- Macro-Recall: 92.6%
- Macro-F1: 92.4%

Why macro metrics: Each class contributes equally to the summary score, even in balanced datasets.

Takeaway: The model learns features that generalise across all classes with strong accuracy-per-parameter balance.

Slide 12 — Lessons Learned and Next Steps

Augmentation strength has a clear effect on generalisation; heavy transforms can harm fine-grained classes. Regularisation stabilises training, yet gains taper after a point. Keep weight decay and dropout modest. Cosine learning-rate schedule converged smoothly; early stopping prevented overfitting. Next steps: try label smoothing and mixup/cutmix; assess calibration and confidence thresholds for safer decisions.

Slide 13 & 14 — Closing and references

Closing: A compact residual design delivers strong results on small images with modest compute. The combination of careful augmentation, cosine decay, and SGD-momentum produced stable convergence and a narrow generalisation gap. Next steps: Explore label smoothing, mix up/cutmix, or stochastic depth; profile CPU inference for deployment trade-offs.

Key references:

Deng, J. et al. (2009) ‘ImageNet: A large-scale hierarchical image database’, CVPR, pp. 248–255.

Goodfellow, I., Bengio, Y. and Courville, A. (2016) Deep Learning. MIT Press. He, K., Zhang, X., Ren, S. and Sun, J. (2016) ‘Deep residual learning for image recognition’, CVPR, pp. 770–778.

Kingma, D.P. and Ba, J. (2015) ‘Adam: A method for stochastic optimization’, ICLR (arXiv:1412.6980).

Krizhevsky, A. (2009) ‘Learning multiple layers of features from tiny images’, Technical Report, University of Toronto.

Pedregosa, F. et al. (2011) ‘Scikit-learn: Machine Learning in Python’, JMLR, 12, pp. 2825–2830.

Shorten, C. and Khoshgoftaar, T.M. (2019) ‘A survey on image data augmentation for deep learning’, Journal of Big Data, 6(60), pp. 1–48.

Simonyan, K. and Zisserman, A. (2015) ‘Very deep convolutional networks for large-scale image recognition’, ICLR (arXiv:1409.1556).

Smith, L.N. (2017) ‘Cyclical learning rates for training neural networks’, WACV, pp. 464–472.

Tan, M. and Le, Q. (2019) ‘EfficientNet: Rethinking model scaling for convolutional neural networks’, ICML, pp. 6105–6114.

TensorFlow-Keras (n.d.) ‘CIFAR-10 dataset’, Available at: https://keras.io/api/datasets/cifar10/ (Accessed 10 October 2025).

University of Toronto (n.d.) ‘CIFAR-10’, Available at: https://www.cs.toronto.edu/~kriz/cifar.html (Accessed 10 October 2025).